Most Important concepts to keep in Mind

- **Variables –** Store data using var, let, and const.
- **Functions –** Reusable blocks of code that perform specific tasks.
- **Hoisting –** JavaScript moves declarations to the top before execution.
- **Closures –** A function inside another function that remembers outer variables.
- **Promises & Async/Await –** Handle asynchronous operations efficiently.
- **DOM Manipulation –** Interact with and update HTML elements dynamically.
- **Event Handling –** Respond to user actions like clicks and keypresses.
- **Prototype & Inheritance –** Helps in object-oriented programming in JavaScript.

➢ How do you merge two strings in JavaScript?

Ans.

In JavaScript, strings can be concatenated (joined) using either the + operator or template literals introduced in ES6.

- Using the + operator:

```
let firstName = "Prep";
let lastName = "Insta";
let fullName = firstName + lastName;
console.log(fullName); // Output: PrepInsta
```

Using Template Literals (Preferred for readability):

```
let fullName = `${firstName} ${lastName}`;
console.log(fullName); // Output: Prep Insta
```

- ## What will be the result of 4 + 1 + "9" in JavaScript?

Ans.

The output will be “59” due to JavaScript’s type coercion rules.

- Step-by-step Execution:
  - 4 + 1 is evaluated first: it returns 5 (a number).
  - Then 5 + “9” is evaluated. JavaScript converts 5 to a string (“5”) and performs string concatenation, resulting in “59”.

```
console.log(4 + 1 + "9"); // Output: "59"
```

Ques 3:How is TypeScript better than JavaScript?

Ans.

TypeScript is a superset of JavaScript that introduces static typing, interfaces, and compile-time checking.

Key Differences:

- Static Typing:
  - TypeScript allows developers to define data types, reducing runtime errors.

```
let age: number = 25;
```

- Interfaces and Classes:
  - Encourages object-oriented programming with clearer structure.
- Better Tooling:
  - Enables IDE support with features like autocompletion, navigation, and error checking.

TypeScript compiles to JavaScript, ensuring compatibility while improving code maintainability in large-scale applications.

Ques 4:What is variable scope, and how does it work in JavaScript?

Ans.

Scope refers to the visibility and accessibility of variables within the code. In JavaScript, there are three types of scope:

1. **Global Scope** – Available everywhere in the program.
2. **Function Scope** – Accessible only inside a function.
3. **Block Scope** – (Introduced with let and const) Variables inside {}.

```
function testScope() {
  if (true) {
    var x = 10;      // function-scoped
    let y = 20;      // block-scoped
    const z = 30;    // block-scoped
  }
  console.log(x); // Works
  // console.log(y); // Error
  // console.log(z); // Error
}
testScope();
```

Ques 5:Explain the concept of prototypal inheritance.

Ans.

In JavaScript, every object has an internal link to another object called its prototype. When you try to access a property that doesn't exist on the current object, JavaScript looks up the chain (prototype chain) until it finds it or reaches null.

```
const parent = {
  greet() {
    return "Hello from parent!";
  }
};

const child = Object.create(parent);
console.log(child.greet()); // Output: Hello from parent!
```

This is known as prototypal inheritance, and it differs from classical inheritance (like in Java). It allows for flexible and memory-efficient object reuse.

Ques 6:What is the difference between deep copy and shallow copy in JavaScript?

Ans.

- A shallow copy copies references to nested objects, not the actual objects.
- A deep copy creates completely independent copies of all objects.

Example:

```
let original = { a: 1, b: { c: 2 } };
let shallowCopy = { ...original };
let deepCopy = JSON.parse(JSON.stringify(original));

original.b.c = 42;
console.log(shallowCopy.b.c); // 42 (affected)
console.log(deepCopy.b.c);    // 2  (unaffected)
```

Shallow copies are faster but can lead to bugs when dealing with nested data. Deep copies are safer but more expensive in terms of performance.

Ques 7:What is the use of setTimeout() and setInterval()?

Ans.

Both are used to schedule the execution of code:

- setTimeout(fn, delay) – Runs fn once after delay milliseconds.
- setInterval(fn, delay) – Repeats fn every delay milliseconds.

```
setTimeout(() => console.log("Runs after 2 seconds"), 2000);

let counter = 0;
let timer = setInterval(() => {
  counter++;
  console.log(counter);
  if (counter === 3) clearInterval(timer);
}, 1000);
```

These are key in building timers, animations, polling, or delaying actions.

Ques 8:What does the bind() method do in JavaScript?

Ans.

The bind() method creates a new function with a specific this value and optional initial arguments.

```
const user = {
  name: "Alice",
  greet() {
    console.log(`Hello, ${this.name}`);
  }
};

const greetFn = user.greet.bind(user);
greetFn(); // Hello, Alice
```

It’s useful when passing object methods as callbacks or event handlers where the context may otherwise be lost.

Ques 9:How do you handle exceptions in JavaScript?

Ans.

JavaScript uses try…catch…finally to handle errors gracefully and avoid program crashes.

```
try {
  // Code that may throw an error
  throw new Error("Something went wrong!");
} catch (err) {
  console.error("Caught error:", err.message);
} finally {
  console.log("This always runs");
}
```

- **try** – Wraps code that might throw an error.
- **catch** – Handles the error.
- **finally** – Executes regardless of whether an error occurred.

This structure ensures robust and fault-tolerant code.

Ques 10:Is JavaScript the same as Java?

Ans.

No, JavaScript and Java are completely different languages despite the similarity in names. The naming was marketing-driven.

Aspect	Java	JavaScript
Type	Compiled, statically typed	Interpreted, dynamically typed
Usage	Backend, Android, enterprise	Web development (frontend & backend via Node.js)
Syntax Similarity	C-style syntax	C-style syntax

JavaScript Interview Questions and Answers for Freshers

Ques 1: What is JavaScript? Why do we need it?

Ans.

JavaScript is a high-level, interpreted programming language primarily used to create interactive and dynamic content on web pages.

- It enables developers to implement complex features such as real-time updates, interactive forms, animations, and more.
- Unlike HTML and CSS, which structure and style web content respectively, JavaScript adds behavior to web pages, allowing for user engagement and dynamic content updates without requiring a page reload.

Ques 2: What are the different data types in JavaScript?

Ans.

JavaScript supports several data types, categorized into:

Primitive Types: These include:

- **String:** Represents textual data. Example: “Hello, World!”
  Number: Represents numerical values. Example: 42 or 3.14
- **Boolean:** Represents logical values. Example: true or false
- **Null:** Represents the intentional absence of any object value. Example: null
- **Undefined:** Indicates a variable that has been declared but not assigned a value. Example: undefined
- **Symbol:** Represents a unique and immutable value, often used as object property identifiers.
- **BigInt:** Represents integers with arbitrary precision. Example: 1234567890123456789012345678901234567890n

Non-Primitive Types:

- **Object:** A collection of key-value pairs. Objects can represent more complex data structures like arrays, functions, dates, and regular expressions.

Understanding these data types is crucial for effective variable management and function operations in JavaScript.

Ques 3: What is the difference between var, let, and const?

Ans.

In JavaScript, **variables can be declared using var, let, or const,** each with distinct characteristics:

var:

- **Scope:** Function-scoped. If declared outside a function, it becomes globally scoped.
- **Hoisting:** Declarations are hoisted to the top of their scope and initialized with undefined.
- **Re-declaration and Update:** Can be re-declared and updated within its scope.

let:

- **Scope:** Block-scoped, confined to the block (enclosed by {}) where it's declared.
- **Hoisting:** Declarations are hoisted but not initialized, leading to a "temporal dead zone" until the declaration is encountered.
- **Re-declaration and Update:** Cannot be re-declared within the same scope but can be updated.

const:

- **Scope:** Block-scoped, similar to let.
- **Hoisting:** Behaves like let with hoisting.
- **Re-declaration and Update:** Cannot be re-declared or updated within the same scope. However, for objects, while the variable reference cannot be changed, the object's properties can be modified.

Example :

```
function example() {
if (true) {
var x = 1;
let y = 2;
const z = 3;
console.log(x); // Outputs: 1
console.log(y); // Outputs: 2
console.log(z); // Outputs: 3
}
console.log(x); // Outputs: 1 (var is function-scoped)
// console.log(y); // Error: y is not defined (let is block-
scoped)
// console.log(z); // Error: z is not defined (const is block-
scoped)
}
example();
```

- **Note –** Choosing the appropriate declaration keyword is essential for managing variable scope and ensuring code maintainability.

Ques 4: What is a closure in JavaScript?

Ans.

A closure is a function that retains access to its lexical scope, even when the function is executed outside that scope.

- This means the inner function has access to variables and parameters of its outer function, even after the outer function has finished executing.
- Closures are often used to create private variables or functions.

Example:

```
function outerFunction() {
let count = 0;
return function innerFunction() {
count++;
return count;
};
}

const increment = outerFunction();
console.log(increment()); // Outputs: 1
console.log(increment()); // Outputs: 2
```

- In this example, **innerFunction** forms a closure, allowing it to access and modify the count variable defined in **outerFunction's** scope, even after outerFunction has completed execution.

Ques 5: Explain event delegation in JavaScript.

Ans.

Event delegation is a technique where a **single event listener is added to a parent element to manage events for its child elements.**

- Instead of attaching individual event listeners to each child element, the parent element listens for events, and through event propagation (bubbling), it can capture events from its descendants.
- This approach enhances performance and simplifies code management, especially when dealing with dynamic content.

Example:

```
document.getElementById('parent').addEventListener('click',
function(event) {
if (event.target && event.target.matches('button.className')) {
console.log('Button clicked:', event.target);
}
});
```

- In this example, a **click event listener** is added to the parent element with the ID parent. When any button with the class **className** inside this parent is clicked, the event is captured by the parent, and the specified action is executed.
-

Ques 6: What is the difference between == and === in JavaScript? (Most important Question)

Ans.

In JavaScript, **== and ===** are comparison operators with distinct behaviors:

== (Loose Equality):

- Compares two values for equality after converting both values to a common type **(type coercion)**.
- **Example:** 5 == '5' returns true because the string '5' is converted to the number 5 before comparison.

=== (Strict Equality):

- Compares both the value and the type without performing type conversion.
- **Example:** 5 === '5' returns false because the types (number and string) are different.

Note - Using === is generally recommended to avoid unexpected results from type coercion.

Ques 7: What is the purpose of the this keyword in JavaScript?

Ans.

The this keyword refers to the object from which a function was called. Its value depends on the context in which the function is executed:

- **Global Context:** In the global scope, this refers to the global object (window in browsers).
- **Function Context:** Inside a regular function, this refers to the global object in non-strict mode and undefined in strict mode.
- **Method Context:** When a function is called as a method of an object, this refers to the object itself.
- **Constructor Context:** In a constructor function, this refers to the newly created instance.

Example:

```
const obj = {
name: 'Alice',
greet: function() {
console.log(`Hello, ${this.name}`);
}
};

obj.greet(); // Outputs: Hello, Alice
```

- In this example, within the greet method, the **this keyword** refers to obj, so this.name returns 'Alice'.

Tip 2 : Practice Coding Problems Solve problems on arrays, strings, recursion, and algorithms on platforms like LeetCode to improve problem-solving skills.

Ques 8: What is the difference between function declaration and function expression?

Ans.

In JavaScript, functions can be defined in two primary ways:

- **Function Declaration:** A function is declared using the function keyword and can be called before its definition due to hoisting.

```
function greet() {
console.log("Hello!");
}
greet(); // Outputs: Hello!
```

Function Expression: A function is assigned to a variable. It is not hoisted, so it cannot be called before the definition.

```
const greet = function() {
console.log("Hello!");
};
greet(); // Outputs: Hello!
```

- Function expressions are commonly used when assigning functions to variables, passing them as arguments, or using them in closures.

Ques 9: What are Arrow Functions in JavaScript?

Ans.

Arrow functions, introduced in **ES6**, provide a more concise syntax for writing functions. They do not have their own this, instead inheriting this from their surrounding lexical scope.

const add = (a, b) => a + b;
console.log(add(5, 3)); // Outputs: 8

Differences from Regular Functions:

- Do not bind their own this, making them useful in callback functions.
- Cannot be used as constructors (new cannot be used with arrow functions).
- Do not have an arguments object.

Arrow functions are particularly useful for short, one-line functions or when working with callbacks.

Ques 10: What is an Immediately Invoked Function Expression (IIFE)?

Ans.

An IIFE (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined. It prevents polluting the global scope by encapsulating variables inside the function.

```
(function() {
console.log("IIFE is executed!");
})();
```

or using the arrow function:

```
(() => {
console.log("IIFE is executed!");
})();
```

- IIFEs are often used to create private variables, initialize modules, or execute a block of code only once.

JavaScript Interview Questions and Answer for Experienced

Ques 11: What is the difference between null and undefined?

Ans.

- **null:** It is an intentional absence of value, meaning a variable has been explicitly set to "nothing."

```
let value = null;
console.log(value); // Outputs: null
```

- **undefined:** It means a variable has been declared but has not been assigned a value.

```
let value;
console.log(value); // Outputs: undefined
```

- While both indicate missing values, null is assigned intentionally, whereas undefined happens by default.

Tip 3 : Learn Asynchronous JavaScript Be confident with Promises, async/await, and event loops since they are commonly asked in interviews.

Ques 12: What are template literals in JavaScript?

Ans.

Template literals (introduced in ES6) are enclosed in backticks (` `) and allow:

- Multiline strings
- String interpolation
- Embedded expressions

Example:

```
const name = "Alice";
const message = `Hello, ${name}!`;
console.log(message); // Outputs: Hello, Alice!
```

- They make it easier to work with strings compared to traditional concatenation (+).

Ques 13: What is the difference between map(), filter(), and reduce()?

Ans.

These are array methods that manipulate elements:

- **map():** Creates a new array by applying a function to each element.

```
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Outputs: [2, 4, 6]
```

- **filter():** Returns a new array with elements that pass a test.

```
const numbers = [1, 2, 3, 4];
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // Outputs: [2, 4]
```

- **reduce():** Reduces an array to a single value.

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((acc, num) => acc + num, 0);
console.log(sum); // Outputs: 10
```

Each method is useful for working with arrays in functional programming.

Ques 14: What is destructuring in JavaScript?

Ans.

Destructuring is a feature that allows extracting values from arrays or objects into variables.

Array destructuring:

```
const numbers = [1, 2, 3];
const [a, b, c] = numbers;
console.log(a, b, c); // Outputs: 1 2 3
```

Object destructuring:

```
const person = { name: "Alice", age: 25 };
const { name, age } = person;
console.log(name, age); // Outputs: Alice 25
```

It simplifies variable assignment and improves code readability.

Tip 4: Understand DOM & Event Handling Know how to manipulate HTML elements, handle events (click, keypress), and work with forms in JavaScript.

Ques 15: What is the difference between synchronous and asynchronous JavaScript?

Ans.

- **Synchronous JavaScript:** Executes code line-by-line, blocking further execution until the current operation completes.

```
console.log("Start");
console.log("End");
// Outputs: Start, then End
```

- **Asynchronous JavaScript:** Uses callbacks, promises, or async/await to execute code without blocking.

```
console.log("Start");
setTimeout(() => console.log("Middle"), 1000);
console.log("End");
// Outputs: Start, End, then Middle (after 1 second)
```

Asynchronous programming is essential for handling API calls and user interactions.

Ques 16: What are Promises in JavaScript?

Ans.

A Promise represents a future value and is used to handle asynchronous operations.

States of a Promise:

- **Pending:** Initial state, before the operation completes.
- **Fulfilled:** The operation was successful.
- **Rejected:** The operation failed.

Example:

```
const fetchData = new Promise((resolve, reject) => {
setTimeout(() => resolve("Data received"), 2000);
});

fetchData.then(response => console.log(response)); // Outputs:
Data received (after 2 seconds)
```

Promises help manage asynchronous code more effectively.

Ques 17: What is async/await in JavaScript?

Ans.

async/await is a syntactic sugar over Promises, making asynchronous code easier to read and write.

Example:

```
async function fetchData() {
let data = await new Promise(resolve => setTimeout(() =>
resolve("Data received"), 2000));
console.log(data);
}

fetchData(); // Outputs: Data received (after 2 seconds)
```

It helps avoid callback hell and improves code readability.

Ques 18: What is the event loop in JavaScript?

Ans

- **object-fit:** Defines how an image fits inside its container.

```
img {
object-fit: cover;
}
```

- **object-position:** Defines the position of the image inside its container.

```
img {
object-position: center;
}
```

Ques 19: What is the typeof operator?

Ans.

The **typeof** operator is used to determine the type of a given operand. It returns a string indicating the type of the operand, such as number, string, boolean, etc.

```
console.log(typeof 42); // Output: number
console.log(typeof 'hello'); // Output: string
console.log(typeof true); // Output: boolean
console.log(typeof undefined); // Output: undefined
console.log(typeof null); // Output: object (this is a known bug in
JavaScript)
```


Ques 20: What is the spread operator in JavaScript?

Ans.

The **spread operator (…)** allows you to unpack elements from an array or object and spread them into a new array or object. It is often used for copying or merging arrays/objects.

```
// Spread in Arrays
const arr = [1, 2, 3];
const newArr = [...arr, 4, 5];
console.log(newArr); // Output: [1, 2, 3, 4, 5]

// Spread in Objects
const obj = { name: 'Alice', age: 25 };
const newObj = { ...obj, city: 'New York' };
console.log(newObj); // Output: { name: 'Alice', age: 25, city: 'New York'
}
```

Ques 21: What is event bubbling in JavaScript?

Ans.

Event bubbling refers to the way events propagate in the DOM hierarchy. When an event is triggered on an element, it bubbles up to the parent elements until it reaches the root of the document, unless explicitly stopped.

```
document.getElementById('child').addEventListener('click', function() {
console.log('Child clicked!');
});

document.getElementById('parent').addEventListener('click', function() {
console.log('Parent clicked!');
});
```

If the child element is clicked, the "Child clicked!" message will be logged first, followed by "Parent clicked!" due to event bubbling.

Ques 22: Explain the concept of hoisting in JavaScript.

Ans.

Hoisting is a behavior in JavaScript where variable and function declarations are moved to the top of their containing scope during the compile phase. However, only the declarations are hoisted, not the assignments.

- **Variables:** Only the declaration (var x;) is hoisted, not the assignment (x = 10;).
- **Functions:** Both function declarations and their definitions are hoisted, but function expressions are not.

Example:

```
console.log(a); // Output: undefined
var a = 10;

console.log(b); // ReferenceError: b is not defined
let b = 20;

foo(); // Output: Hello
function foo() {
console.log('Hello');
}
```